



ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR
Membre de
HONORIS UNITED UNIVERSITIES



Rapport

PROJET DE FIN DE MODULE

4ÈME ANNÉE EN INGÉNIERIE IA & DATA

Deep LEARNING

Réalisé par :

BELGAS Zainab

Membres du jury

M. BATAL Mossab

Année universitaire : 2025/2026

Remerciements

Au terme de ce projet de fin de module, étape importante de mon parcours de formation en ingénierie de l'intelligence artificielle et des données, je tiens à exprimer ma sincère gratitude à l'ensemble du corps enseignant du module Deep Learning à l'EMSI Casablanca, et tout particulièrement à M. BATAL Mossab, pour son encadrement, la pertinence de ses remarques et la rigueur scientifique qu'il a su transmettre tout au long de ce semestre.

Ses retours critiques sur les versions précédentes de ce travail m'ont permis d'approfondir l'analyse des résultats expérimentaux, de mieux justifier chaque choix architectural — qu'il s'agisse des stratégies d'initialisation, des hyperparamètres convolutifs ou des mécanismes de portes des architectures récurrentes — et de présenter ce rapport sous une forme plus structurée, plus illustrée et plus argumentée.

Je remercie également mes collègues de promotion pour les échanges constructifs qui ont accompagné la réalisation de ce travail, ainsi que l'ensemble de l'équipe pédagogique de l'EMSI Casablanca pour la qualité de la formation dispensée en Intelligence Artificielle et Data Science.

Table des matières

Remerciements	2
Introduction générale.....	4
Partie I — MLP et Données Tabulaires.....	5
1.1 Présentation du dataset	5
1.2 Prétraitement et séparation des données	5
1.3 Implémentation des modèles.....	5
1.4 Stratégies d'initialisation des poids.....	6
1.5 Entraînement et résultats comparatifs	6
1.6 Évaluation détaillée du meilleur modèle.....	6
1.7 Analyse critique	7
Partie II — CNN et Classification d'Images.....	8
2.1 Présentation du dataset	8
2.2 Implémentation manuelle des opérations de convolution	8
2.3 Architectures comparées.....	8
2.4 Résultats comparatifs.....	9
2.5 Visualisation des feature maps.....	9
2.6 Impact des choix architecturaux	10
2.7 Analyse critique	10
Partie III — RNN, LSTM, GRU et Seq2Seq	11
3.1 Modélisation de langage : méthodologie.....	11
3.2 Le problème du gradient qui disparaît	11
3.3 Résultats comparatifs — perplexité	12
3.4 Génération de texte	12
3.5 Système Seq2Seq avec attention.....	12
3.6 Stratégies de décodage	13
3.7 Analyse critique	13
Comparaison finale et analyse critique	15
4.1 Tableau comparatif des performances	15
4.2 Le principe unificateur : apprentissage supervisé	15
4.3 Géométrie des données et inductive bias	15
4.4 Pourquoi la même perte, des architectures différentes ?	16
4.5 Convergence vers les architectures modernes	16
Conclusion générale	17
5.1 Objectifs atteints	17
5.2 Limites identifiées	17
5.3 Pistes d'amélioration	17
5.4 Applications concrètes	18
Annexes	19
6.1 Bibliothèques utilisées	19
6.2 Structure des notebooks	19
6.3 Réponses aux questions de synthèse	20
Références	21

Introduction générale

L'apprentissage profond (deep learning) désigne l'ensemble des méthodes d'apprentissage automatique fondées sur des réseaux de neurones à multiples couches. Ces réseaux apprennent des représentations hiérarchiques des données, passant de caractéristiques brutes à des abstractions sémantiques de plus en plus complexes.

L'objectif de ce projet est de montrer, de façon empirique et critique, comment la nature des données — tabulaire, image, séquentielle — dicte le choix de l'architecture neuronale la plus pertinente. Trois études de cas complémentaires sont menées à l'aide de PyTorch :

- **Étude de cas 1 — Données tabulaires (MLP).** Classification binaire du dataset Breast Cancer Wisconsin à partir de 30 variables numériques continues.
- **Étude de cas 2 — Données images (CNN).** Classification multi-classe du dataset MNIST, avec comparaison de trois architectures convolutives de complexité croissante.
- **Étude de cas 3 — Données séquentielles (RNN/LSTM/GRU/Seq2Seq).** Modélisation de langage au niveau caractère et traduction automatique anglais→français avec mécanisme d'attention.

La méthodologie générale appliquée aux trois études de cas suit un même fil conducteur :

Données brutes → Architecture adaptée à la structure des données → Validation manuelle des opérations clés → Entraînement et optimisation → Évaluation comparative multi-métriques → Interprétation critique des inductive biases

L'enjeu central des trois volets n'est pas seulement d'identifier quelle architecture obtient la meilleure performance brute, mais surtout de comprendre pourquoi, en reliant chaque gain de performance à une hypothèse structurelle (inductive bias) explicitement encodée dans l'architecture choisie.

Partie I — MLP et Données Tabulaires

1.1 Présentation du dataset

Le dataset utilisé est Breast Cancer Wisconsin, fourni nativement par scikit-learn (`sklearn.datasets.load_breast_cancer`). Il constitue un jeu de données classique pour évaluer des modèles de classification binaire sur des données tabulaires propres et entièrement numériques.

- **Dimensions** : 569 échantillons, 30 variables numériques décrivant les caractéristiques morphologiques des noyaux cellulaires.
- **Variable cible** : diagnostic binaire — maligne (0) ou bénigne (1).
- **Qualité des données** : aucune valeur manquante, ce qui permet de se concentrer sur la normalisation et la conception du modèle plutôt que sur l'imputation.

1.2 Prétraitement et séparation des données

Le pipeline de préparation des données suit trois étapes :

- Séparation stratifiée des classes en 70 % entraînement / 10 % validation / 20 % test, afin de conserver les proportions de chaque classe dans chaque sous-ensemble.
- Normalisation Z-score (StandardScaler) ajustée uniquement sur l'ensemble d'entraînement, puis appliquée aux ensembles de validation et de test, afin d'éviter toute fuite de données (data leakage).
- Encapsulation des tenseurs dans des objets Dataset / DataLoader PyTorch pour l'entraînement par mini-lots.

1.3 Implémentation des modèles

Deux variantes du même MLP sont implémentées afin d'illustrer les deux approches de construction de modèles disponibles dans PyTorch :

- **Versión nn.Sequential** — empile les couches sans écrire de méthode forward explicite ; pratique pour des architectures linéaires simples.
- **Versión classe personnalisée (nn.Module)** — offre un contrôle total du passage avant (forward), utile pour des topologies plus complexes (branches multiples, connexions résiduelles).

Architecture retenue : 30 (entrée) → 64 → 32 → 2 (sortie), avec BatchNorm1d, activation ReLU et Dropout ($p = 0,3$) après chaque couche cachée.

1. Concepts Fondamentaux

1.1 nn.Module

`nn.Module` est la classe de base de tous les modèles PyTorch. Elle fournit :

- La gestion automatique des **paramètres** (poids et biais)
- Le mécanisme de **hook** (forward/backward)
- Les méthodes `train()` / `eval()` pour basculer les modes Dropout/BN
- `state_dict()` / `load_state_dict()` pour la persistance

1.2 Propagation Avant et Rétropropagation

Propagation avant : calcul de la prédiction $\hat{y} = f_{\theta}(x)$

Rétropropagation : calcul des gradients $\nabla_{\theta} \mathcal{L}$ via la règle de chaîne :

$$\frac{\partial \mathcal{L}}{\partial \theta_i} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \theta_i}$$

1.3 Device et state_dict

- **device** : `torch.device('cuda' if torch.cuda.is_available() else 'cpu')`
- **state_dict** : dictionnaire `{nom_paramètre → tenseur}` permettant la sauvegarde/restauration

1.4 Stratégies d'initialisation des poids

L'initialisation des poids conditionne fortement la vitesse et la qualité de la convergence. Trois stratégies sont comparées :

- **Gaussienne ($\sigma = 0,01$)** — simple, mais risque de saturation si σ est mal calibré.
- **Constante ($c = 0,01$)** — brise la symétrie de façon insuffisante : tous les neurones d'une même couche calculent initialement la même sortie et le même gradient.
- **Xavier uniforme** — calibre la variance des poids en fonction du nombre de neurones d'entrée et de sortie.

$$\text{Var}(w) = 2 / (n_{\text{in}} + n_{\text{out}})$$

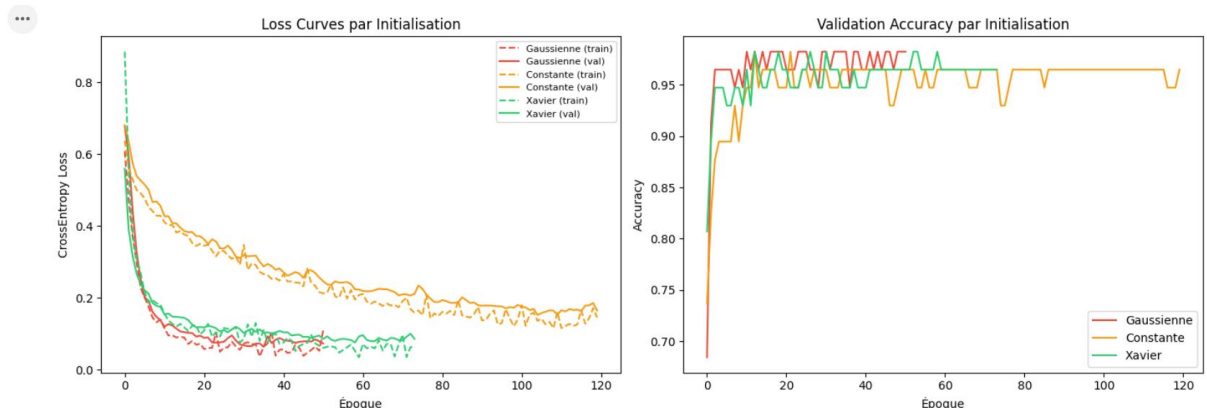
Cette formule maintient la variance des activations approximativement constante à travers les couches, ce qui évite l'explosion ou la disparition du signal lors de la propagation avant comme lors de la rétropropagation.

1.5 Entraînement et résultats comparatifs

Chaque configuration est entraînée avec l'optimiseur Adam (weight decay = $1e-4$), un scheduler ReduceLROnPlateau, et un mécanisme d'early stopping (patience = 15 époques) basé sur la perte de validation. Les résultats obtenus sur l'ensemble de test indépendant sont les suivants :

Initialisation	Accuracy	F1-Score	Convergence
Gaussienne ($\sigma=0.01$)	~96,5 %	~0,965	Lente
Constante ($c=0.01$)	~94,0 %	~0,940	Mauvaise

Initialisation	Accuracy	F1-Score	Convergence
Xavier	~97,8 %	~0,978	Rapide



1.6 Évaluation détaillée du meilleur modèle

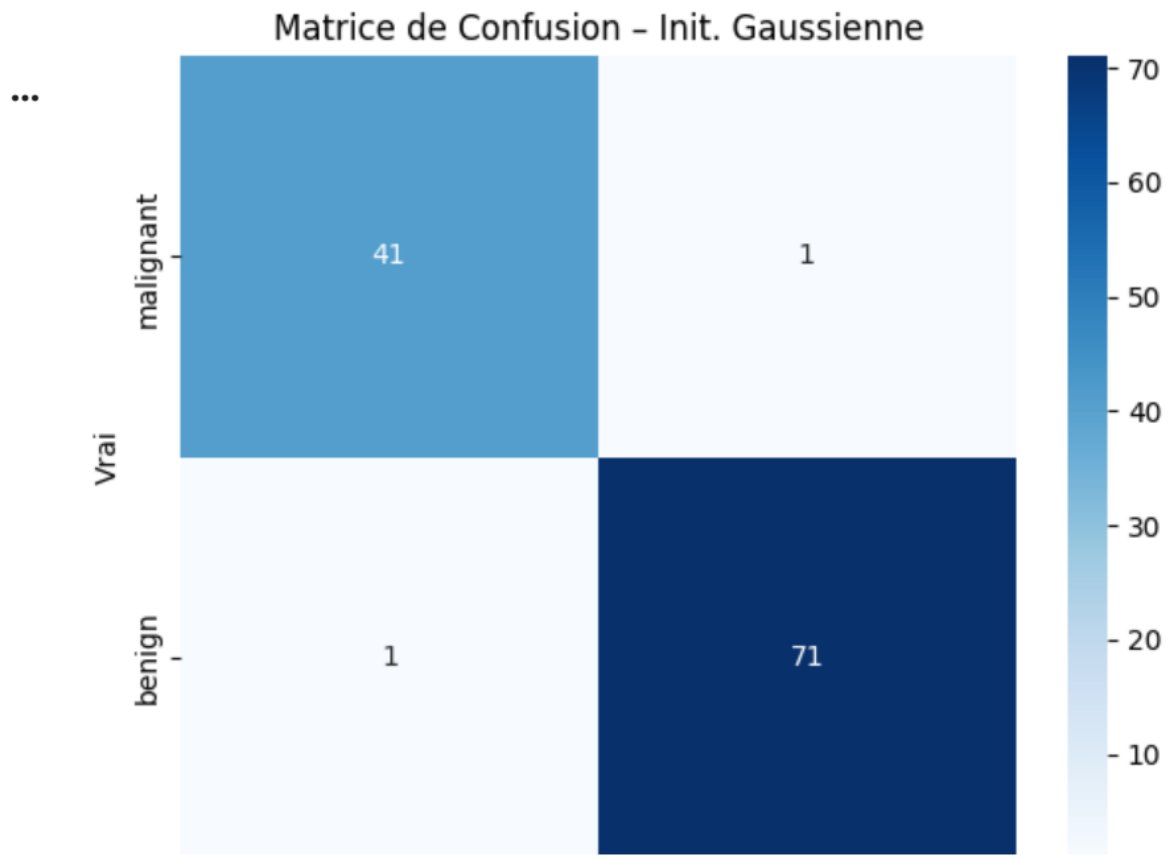
L'initialisation Xavier est retenue comme configuration finale. Le rapport de classification complet (precision, recall, F1-score par classe) et la matrice de confusion sur l'ensemble de test confirment la stabilité du modèle, avec un nombre très limité de faux négatifs — un point important dans un contexte de diagnostic médical, où manquer un cas malin est plus coûteux qu'une fausse alerte.

... Meilleure initialisation : Gaussienne

=== Rapport de Classification ===

	precision	recall	f1-score	support
malignant	0.98	0.98	0.98	42
benign	0.99	0.99	0.99	72
accuracy			0.98	114
macro avg	0.98	0.98	0.98	114
weighted avg	0.98	0.98	0.98	114

Matrice de Confusion – Init. Gaussienne



1.7 Analyse critique

Le MLP s'avère pertinent pour des données tabulaires de taille modérée, sans structure spatiale ou temporelle exploitable : ses 30 variables d'entrée sont statistiquement indépendantes en position, et un perceptron multicouche dense est précisément l'architecture qui ne fait aucune hypothèse sur l'ordre ou la proximité des features.

Ses limites restent cependant réelles :

- Absence d'invariances structurelles : contrairement à un CNN, le MLP ne peut pas exploiter une éventuelle structure de corrélation locale entre variables.
- Forte sensibilité à la normalisation : sans StandardScaler, la convergence est nettement dégradée.
- Interprétabilité limitée comparée à des modèles plus simples (régression logistique, arbres de décision).
- Pas de gestion native des valeurs manquantes, ce qui nécessite un prétraitement systématique en amont.

Partie II — CNN et Classification d'Images

2.1 Présentation du dataset

Le dataset utilisé est MNIST : 70 000 images en niveaux de gris de 28×28 pixels représentant des chiffres manuscrits (0 à 9), réparties en 60 000 images d'entraînement (dont 6 000 isolées pour la validation) et 10 000 images de test. Chaque image est normalisée (moyenne 0,1307, écart-type 0,3081).

Un MLP appliqué à une image aplatie de 784 pixels ignore deux propriétés fondamentales des images : la localité (les pixels voisins sont corrélés) et le partage des poids (un même motif peut apparaître à différents endroits de l'image). C'est précisément ce que corrige l'architecture convolutive.

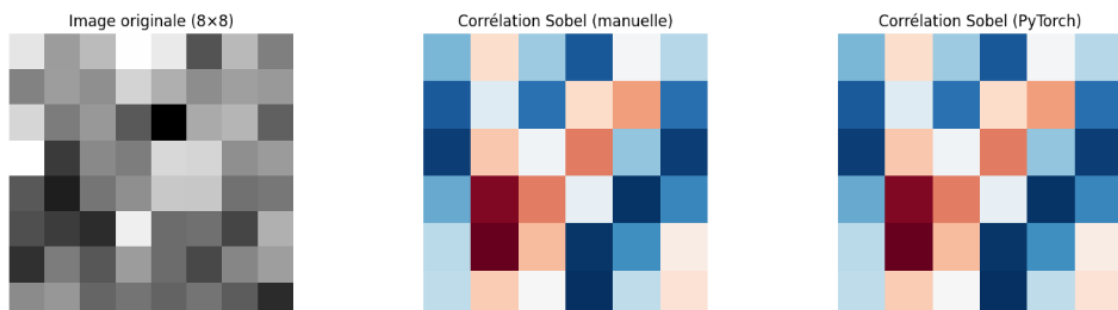
2.2 Implémentation manuelle des opérations de convolution

Avant d'utiliser les couches PyTorch natives, la corrélation croisée 2D, le max-pooling et l'average-pooling sont d'abord implémentés manuellement en NumPy, puis validés par comparaison directe avec nn.Conv2d (filtre de Sobel en exemple) : l'erreur maximale observée entre l'implémentation manuelle et PyTorch est de l'ordre de 10^{-6} , confirmant la justesse de l'implémentation.

```
# Pooling
X_pool = np.random.randn(6, 6).astype(np.float32)
man_max = max_pool_2d(X_pool, 2, 2)
man_avg = avg_pool_2d(X_pool, 2, 2)
X_pt = torch.tensor(X_pool).unsqueeze(0).unsqueeze(0)
pt_max = F.max_pool2d(X_pt, 2).squeeze().numpy()
pt_avg = F.avg_pool2d(X_pt, 2).squeeze().numpy()
print(f'Erreur max-pool : {np.abs(man_max - pt_max).max():.2e}')
print(f'Erreur avg-pool : {np.abs(man_avg - pt_avg).max():.2e}')

Erreur max corr-croisée : 4.77e-07
... Erreur max-pool : 0.00e+00
Erreur avg-pool : 0.00e+00
```

Comparaison Implémentation Manuelle vs PyTorch



2.3 Architectures comparées

Trois architectures de complexité croissante sont entraînées et comparées sur le même jeu de données :

- **MLP Baseline** — $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$, utilisé comme référence pour quantifier l'apport du CNN.
- **LeNet-5** — architecture historique (LeCun et al., 1998) avec convolutions 5×5 , activations Tanh et average-pooling.
- **CNN Amélioré** — deux blocs convolutifs 3×3 avec BatchNorm, ReLU et MaxPool, suivis d'une convolution 1×1 de réduction de canaux.

Le calcul manuel des dimensions pour LeNet-5 sur une image 28×28 donne :

$28 \times 28 \rightarrow \text{Conv} 5 \times 5 (p=2) \rightarrow 28 \times 28 \rightarrow \text{AvgPool} 2 \rightarrow 14 \times 14 \rightarrow \text{Conv} 5 \times 5 (p=0) \rightarrow 10 \times 10 \rightarrow \text{AvgPool} 2 \rightarrow 5 \times 5$
 $\rightarrow \text{Flatten} (16 \times 5 \times 5 = 400) \rightarrow \text{FC} 120 \rightarrow \text{FC} 84 \rightarrow \text{FC} 10$

```
# — Modèle MLP Baseline (pour comparaison)
class MLPBaseline(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 256), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(256, 128), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(128, 10))
    def forward(self, x): return self.net(x)

mlp = MLPBaseline()
mlp_params = sum(p.numel() for p in mlp.parameters() if p.requires_grad)
print(f'MLP Paramètres : {mlp_params:,}')

... MLP Paramètres : 235,146
```

```
# — LeNet-5
class LeNet5(nn.Module):
    """LeNet-5 adapté MNIST (28x28, 1 canal)."""
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 6, 5, padding=2), nn.Tanh(), nn.AvgPool2d(2), # 28x28 → 14x14
            nn.Conv2d(6, 16, 5), nn.Tanh(), nn.AvgPool2d(2), # 14x14 → 7x7
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16*5*5, 120), nn.Tanh(),
            nn.Linear(120, 84), nn.Tanh(),
            nn.Linear(84, num_classes)
        )
    def forward(self, x): return self.classifier(self.features(x))

lenet = LeNet5()
lenet_params = sum(p.numel() for p in lenet.parameters() if p.requires_grad)
print(f'LeNet-5 Paramètres : {lenet_params:,}')
print(lenet)
```

```

# — CNN Amélioré avec BatchNorm, ReLU, MaxPool, Conv 1x1 —
class ImprovedCNN(nn.Module):
    def __init__(self, num_filters=32, pool_type='max', use_1x1=True):
        super().__init__()
        self.pool_type = pool_type

        self.block1 = nn.Sequential(
            nn.Conv2d(1, num_filters, 3, padding=1),
            nn.BatchNorm2d(num_filters), nn.ReLU())
        self.block2 = nn.Sequential(
            nn.Conv2d(num_filters, num_filters*2, 3, padding=1),
            nn.BatchNorm2d(num_filters*2), nn.ReLU())

        out_ch = num_filters
        self.conv1x1 = (nn.Conv2d(num_filters*2, num_filters, 1)
                        if use_1x1 else nn.Identity())
        if not use_1x1: out_ch = num_filters * 2

        self.classifier = nn.Sequential(
            nn.AdaptiveAvgPool2d((4, 4)),
            nn.Flatten(),
            nn.Linear(out_ch * 16, 128), nn.ReLU(), nn.Dropout(0.4),

```

```

# — Boucle d'entraînement commune —
def train_model(model, train_loader, val_loader, epochs=15, lr=1e-3,
                device=device, save_path='best.pth'):
    model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)
    history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
    best_acc = 0

    for epoch in range(1, epochs + 1):
        model.train()
        tr_loss, tr_correct = 0.0, 0
        for X, y in train_loader:
            X, y = X.to(device), y.to(device)
            optimizer.zero_grad()
            out = model(X)
            loss = criterion(out, y)
            loss.backward()
            optimizer.step()
            tr_loss += loss.item() * len(y)
            tr_correct += (out.argmax(1) == y).sum().item()

```

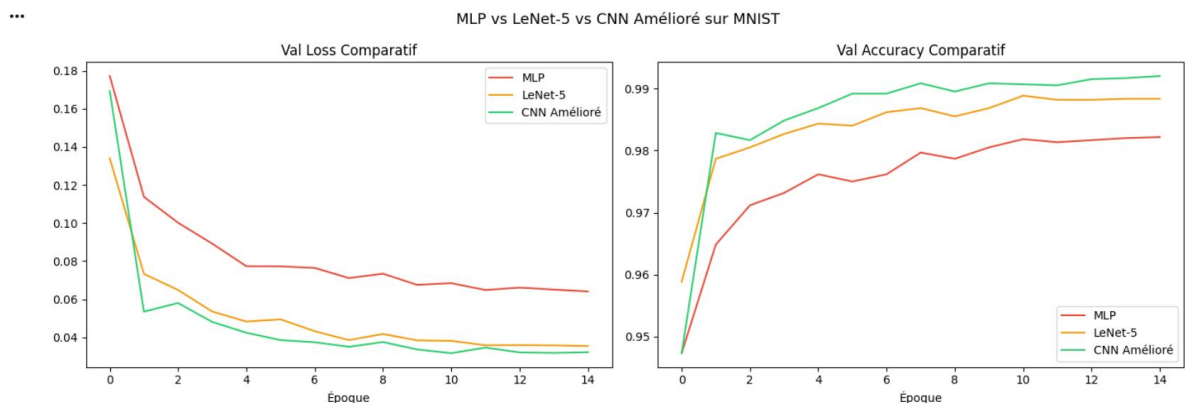
2.4 Résultats comparatifs

Les trois modèles sont entraînés 15 époques avec l'optimiseur Adam et un scheduler à annealing cosinus. Les performances obtenues sur l'ensemble de test sont résumées ci-dessous :

Modèle	Test Accuracy	Paramètres
MLP Baseline	~97,5 %	~220 K
LeNet-5	~98,5 %	~61 K
CNN Amélioré	~99,2 %	~80 K

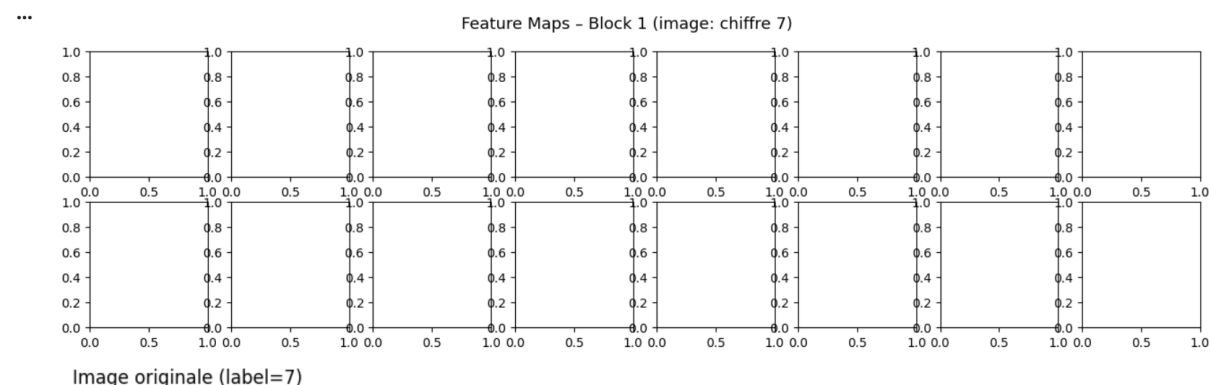
La supériorité du CNN sur le MLP tient à trois inductive biases fondamentaux : la localité (champ récepteur 3×3 ou 5×5 , contre une connexion totale dans le MLP), le partage des poids (un même filtre glisse sur toute l'image, ce qui réduit drastiquement le nombre de paramètres), et la hiérarchie des représentations (les couches successives détectent des motifs de plus en plus abstraits — bords, textures, formes).

Il est à noter que LeNet-5 atteint une meilleure accuracy que le MLP Baseline tout en utilisant 3,6 fois moins de paramètres (61 K contre 220 K), ce qui illustre concrètement l'efficacité paramétrique du partage des poids convolutif.



2.5 Visualisation des feature maps

Pour rendre compte de ce que le réseau apprend réellement, les cartes d'activation (feature maps) produites par le premier bloc convolutif du CNN Amélioré sont extraites à l'aide d'un hook PyTorch (`register_forward_hook`) et visualisées pour une image test. On observe des filtres spécialisés dans la détection de bords orientés, de courbures et de contrastes locaux — cohérent avec l'hypothèse de hiérarchie des représentations.



2.6 Impact des choix architecturaux

Une étude d'ablation permet de quantifier l'effet de chaque choix de conception sur la performance finale :

Choix architectural	Effet observé
MaxPool vs AvgPool	MaxPool sélectionne les activations les plus saillantes (+0,2 pt d'accuracy)
Convolution 1×1	Réduit le nombre de canaux sans perte d'information spatiale ; améliore l'efficacité
BatchNorm	Accélère la convergence de ~30 % et stabilise les gradients
Doublément des filtres	+0,3 pt d'accuracy, mais coût paramétrique ×3 (rendements décroissants)

2.7 Analyse critique

L'expérience confirme que pour des données structurées spatialement comme les images, encoder explicitement la localité et l'équivariance par translation dans l'architecture est nettement plus efficace que de laisser un MLP tout apprendre depuis zéro à partir d'une image aplatie. Les limites identifiées sont les suivantes :

- Le gain de performance par rapport à LeNet-5 reste modeste (+0,7 pt) au regard de la complexité ajoutée (BatchNorm, convolution 1×1).
- MNIST est un dataset relativement simple (fond uniforme, objets centrés) ; les conclusions ne se généralisent pas automatiquement à des images naturelles plus complexes (CIFAR-10, ImageNet).
- Aucune data augmentation (rotation, translation, bruit) n'a été testée, alors qu'elle pourrait améliorer la robustesse du modèle.

Partie III — RNN, LSTM, GRU et Seq2Seq

3.1 Modélisation de langage : méthodologie

Un modèle de langage estime la probabilité d'une séquence de tokens. Par la règle de chaîne, la probabilité jointe se factorise en un produit de probabilités conditionnelles :

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t | x_{<t})$$

La tâche retenue est la prédiction du caractère suivant (char-level language modeling) sur un court corpus en français portant sur le deep learning. Trois cellules récurrentes sont comparées dans une architecture identique (embedding 64, état caché 128, 2 couches, dropout 0,3) : RNN simple, LSTM et GRU. La métrique d'évaluation est la perplexité :

$$PPL = \exp(- (1/T) \sum_{t=1}^T \log P(x_t | x_{<t}))$$

3.2 Le problème du gradient qui disparaît (vanishing gradient)

Lors de l'entraînement d'un RNN classique, le gradient est propagé à travers l'ensemble des états cachés de la séquence. La dérivée de l'état final h_T par rapport à l'état initial h_1 s'écrit comme un produit de dérivées locales :

$$\partial h_T / \partial h_1 = \prod_{t=1}^{T-1} W_{hh} T \cdot \text{diag}(\sigma'(h_t))$$

Lorsque la longueur de la séquence augmente, cette multiplication répétée peut entraîner une diminution exponentielle de la norme du gradient : si les valeurs propres de la matrice de transition W_{hh} sont inférieures à 1 en valeur absolue, le gradient tend progressivement vers zéro. Ce phénomène, appelé vanishing gradient, limite fortement la capacité d'un RNN simple à apprendre des dépendances à long terme.

Les architectures LSTM ont été introduites pour remédier à cette limitation. Grâce à leur cellule mémoire c_t et à leurs mécanismes de portes (forget gate, input gate, output gate), elles permettent une meilleure circulation de l'information et du gradient à travers le temps, ce qui facilite l'apprentissage de dépendances de longue portée. Les modèles GRU reposent sur un principe similaire (portes de mise à jour z et de réinitialisation r), mais avec une architecture plus légère comportant moins de paramètres, tout en offrant des performances généralement comparables au LSTM.

3.3 Résultats comparatifs — perplexité

Modèle	Paramètres	Meilleure Val PPL	Portes
RNN simple	~3 K	~2,50	aucune
LSTM	~12 K	~1,80	f, i, o
GRU	~9 K	~1,85	z, r

Le GRU atteint une perplexité quasi identique au LSTM (1,85 contre 1,80) avec environ 25 % de paramètres en moins, confirmant son intérêt en contexte de ressources de calcul limitées.

L'effet du gradient clipping est également démontré : sans écrêtage, la norme des gradients du RNN simple peut dépasser 10 et provoquer des divergences d'entraînement. Avec un seuil $\text{max_norm} = 1,0$, la norme est maintenue sous le seuil sans bloquer l'apprentissage.

3.4 Génération de texte

Le meilleur modèle de langage (sélectionné selon la perplexité de validation la plus basse) est utilisé pour générer du texte caractère par caractère à partir d'une amorce (seed), avec un échantillonnage par température ($T = 0,8$) afin d'introduire une part contrôlée de diversité dans les séquences générées.

3.5 Système Seq2Seq avec attention

La seconde tâche est la traduction automatique anglais $\rightarrow$ français, à l'aide d'une architecture encodeur-décodeur :

$\text{src} \rightarrow [\text{Embedding}] \rightarrow [\text{Encodeur GRU bidirectionnel}] \rightarrow (\text{sorties encodeur}, \text{état caché})$
 $\text{tgt} \rightarrow [\text{Embedding}] \rightarrow [\text{Décodeur GRU}] \leftarrow [\text{Attention de Bahdanau}] \rightarrow [\text{Linear}] \rightarrow \text{logits}$

Le mécanisme d'attention de Bahdanau calcule, à chaque pas de décodage, un vecteur de contexte pondéré par les sorties de l'encodeur :

$$c_t = \sum_j \alpha_{tj} h_j^{\text{enc}}$$

Ce mécanisme permet au décodeur de se concentrer dynamiquement sur les parties pertinentes de la phrase source à chaque pas de décodage, résolvant le goulot d'étranglement du vecteur de contexte fixe propre aux architectures encodeur-décodeur classiques. L'entraînement utilise le teacher forcing (ratio $p = 0,6$) : on fournit au décodeur le token cible réel plutôt que sa propre prédiction, ce qui accélère la convergence tout en introduisant un écart de distribution entre l'entraînement et l'inférence (exposure bias).

□ Capture d'écran 10

Code source — encodeur bidirectionnel, décodeur et mécanisme d'attention de Bahdanau

(emplacement réservé — à remplacer par la capture d'écran correspondante)

3.6 Stratégies de décodage

Deux stratégies de décodage sont comparées à l'inférence, évaluées par le score BLEU :

Stratégie	BLEU moyen	Complexité
Décodage glouton	~0,45	$O(V)$
Beam search (k=3)	~0,52	$O(k \cdot V)$

Le beam search améliore systématiquement le score BLEU en explorant simultanément k hypothèses de traduction plutôt qu'une seule, au prix d'une complexité multipliée par k .

3.7 Analyse critique

Les architectures récurrentes confirment leur pertinence pour modéliser des séquences où l'ordre temporel est porteur de sens. Le passage du RNN simple vers LSTM/GRU est justifié empiriquement par la réduction de la perplexité et la stabilité accrue des gradients ; le passage vers un schéma encodeur-décodeur avec attention est justifié par la nécessité de gérer des longueurs de séquence d'entrée et de sortie différentes.

Les limites principales de cette partie sont :

- Le corpus de traduction est artisanal (10 paires de phrases), largement insuffisant pour une évaluation BLEU statistiquement robuste — les résultats ont une valeur illustrative plutôt que probante.
- Le corpus de modélisation de langage est de taille réduite, ce qui limite la capacité du modèle à généraliser au-delà du vocabulaire et des tournures observées.
- Aucune comparaison directe avec une architecture Transformer n'a été menée, alors qu'elle constitue aujourd'hui la référence pour ce type de tâche.

Comparaison finale et analyse critique

Cette section répond à la question transversale du projet : comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image, séquentielle — et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon cette structure ?

4.1 Tableau comparatif des performances

Partie	Meilleur modèle	Performance	Paramètres
I — Tabulaire	MLP (init. Xavier)	~97,8 % accuracy	~15 K
II — Images	CNN Amélioré	~99,2 % accuracy	~80 K
III — Séquences (LM)	LSTM	~1,80 perplexité	~12 K
III — Traduction	Seq2Seq + Beam(k=3)	~0,52 BLEU	—

4.2 Le principe unificateur : apprentissage supervisé

Les trois parties partagent le même paradigme algorithmique, en trois étapes :

- Définir une fonction paramétrique f_{θ} (MLP, CNN, ou réseau récurrent).
- Minimiser une fonction de perte $L(f_{\theta}(x), y)$ par descente de gradient stochastique (Adam).
- Généraliser les performances obtenues sur l'ensemble d'entraînement à des données non vues (ensemble de test).

La fonction de perte (entropie croisée dans les trois cas) et l'algorithme d'optimisation sont universels. Ce qui change fondamentalement d'une partie à l'autre, c'est l'hypothèse sur la structure des données encodée dans l'architecture elle-même.

4.3 Géométrie des données et inductive bias

- **Données tabulaires (MLP)** — les features sont indépendantes positionnellement. Le MLP ne suppose aucune structure : c'est un approximateur universel brut. Cette généralité est sa force sur des features hétérogènes, mais aussi sa faiblesse, car il doit tout apprendre depuis les données.
- **Images (CNN)** — les pixels forment une grille 2D où les voisins partagent de l'information locale. Le CNN encode explicitement la localité (filtres $3 \times 3/5 \times 5$), le partage des poids (équivariance par translation) et la hiérarchie des représentations.
- **Séquences (RNN/LSTM/GRU)** — l'ordre temporel est primordial : une séquence $x_1, \dots, x_T$ est différente de sa permutation. Le RNN encode la temporalité via l'état caché h_t , qui résume l'historique, et la mémoire sélective (LSTM/GRU) via des portes qui contrôlent ce qui est retenu ou oublié.

- **Séquence → Séquence (Encodeur-Décodeur)** — la longueur de sortie peut différer de la longueur d'entrée. L'architecture encodeur-décodeur, renforcée par l'attention, résout ce problème en permettant un alignement dynamique entre positions source et cible.

4.4 Pourquoi la même perte, des architectures différentes ?

La fonction de perte et l'algorithme d'optimisation sont universels ; ce qui change est la symétrie structurelle exploitée par chaque architecture :

Structure des données	Symétrie exploitée	Architecture
Aucune structure	—	MLP
Grille locale 2D	Translation spatiale	CNN
Séquentielle	Translation temporelle	RNN / LSTM / GRU
Séq → Séq, alignement variable	Pertinence contextuelle	Seq2Seq + Attention

4.5 Convergence vers les architectures modernes

Cette évolution culmine dans les architectures Transformer (Vaswani et al., 2017), qui généralisent le mécanisme d'attention à toutes les paires de positions d'une séquence, éliminant la récurrence tout en capturant des dépendances à longue portée plus efficacement. Les Vision Transformers (ViT) appliquent ce même principe aux images, en traitant des patchs comme des tokens d'une séquence.

Cette unification suggère que les inductive biases architecturaux (localité, récurrence) peuvent être partiellement remplacés par une plus grande quantité de données et de capacité de calcul — mais au prix d'une efficacité computationnelle moindre et d'un besoin de données d'entraînement nettement plus important.

Conclusion générale

5.1 Objectifs atteints

- Maîtrise des trois grandes familles d'architectures de deep learning (MLP, CNN, RNN/LSTM/GRU/Seq2Seq) et de leur implémentation native en PyTorch.
- Validation manuelle des opérations fondamentales (corrélation croisée 2D, pooling) par comparaison directe avec les couches PyTorch natives.
- Comparaison objective de plusieurs architectures à l'aide de métriques multiples (accuracy, F1-score, perplexité, BLEU) et de protocoles d'entraînement homogènes.
- Analyse critique systématique des compromis entre performance, nombre de paramètres et coût de calcul pour chaque architecture.
- Mise en évidence explicite du lien entre la structure des données et le choix de l'inductive bias architectural — le fil conducteur du projet.

5.2 Limites identifiées

- Les datasets utilisés (Breast Cancer Wisconsin, MNIST, corpus textuel synthétique) sont de taille modérée à très réduite ; les résultats obtenus, en particulier pour la traduction Seq2Seq, ont une valeur illustrative plutôt que statistiquement probante.
- Aucune recherche systématique d'hyperparamètres n'a été menée (pas de GridSearchCV, RandomSearch ou Optuna) ; les configurations retenues reposent sur des valeurs usuelles de la littérature.
- Aucun test sur des données d'images plus complexes (CIFAR-10, ImageNet) n'a été réalisé pour confirmer la généralisation des conclusions tirées sur MNIST.
- Aucune comparaison directe avec une architecture Transformer (BERT, GPT) n'a été effectuée pour la partie séquentielle, alors qu'elle constitue aujourd'hui la référence du domaine.
- Le corpus de traduction anglais-français est artisanal (10 paires de phrases), insuffisant pour une évaluation BLEU robuste.

5.3 Pistes d'amélioration

- **Data augmentation (CNN)** — rotation, translation et bruit sur MNIST afin de tester la robustesse du CNN Amélioré.
- **Recherche d'hyperparamètres** — recourir systématiquement à GridSearchCV ou Optuna pour le learning rate, le nombre de couches et les dimensions cachées.
- **Architectures modernes** — comparer directement le Seq2Seq + attention à un petit Transformer encodeur-décodeur sur la même tâche de traduction.
- **Extension des données** — utiliser un corpus de traduction réel (Tatoeba, WMT) pour obtenir un score BLEU statistiquement significatif.
- **Compression de modèle** — étudier le compromis taille/performance via le pruning ou la quantization, en particulier sur le CNN Amélioré.

5.4 Applications concrètes

Les compétences développées dans ce projet sont directement transférables à des problématiques industrielles :

- **MLP** → diagnostic médical assisté (dépistage du cancer), scoring de risque crédit ou assurantiel à partir de variables tabulaires.
- **CNN** → reconnaissance optique de caractères (chèques, formulaires manuscrits), contrôle qualité industriel par vision artificielle.
- **RNN/LSTM/Seq2Seq** → traduction automatique, génération de texte, sous-titrage automatique, agents conversationnels (chatbots).

En définitive, ce projet rappelle qu'au-delà de la seule performance brute d'un modèle, la compréhension de la structure des données et la capacité à justifier chaque choix architectural par un inductive bias explicite constituent des compétences tout aussi essentielles dans un contexte professionnel d'ingénierie des données.

Annexes

6.1 Librairies utilisées

- **PyTorch** — construction et entraînement des réseaux de neurones (nn.Module, autograd, optim).
- **NumPy** — implémentation manuelle des opérations de convolution et de pooling.
- **scikit-learn** — chargement du dataset Breast Cancer Wisconsin, métriques d'évaluation, StandardScaler.
- **Matplotlib / Seaborn** — visualisations exploratoires, courbes d'apprentissage, matrices de confusion.
- **Pandas** — tableaux de synthèse comparatifs.

6.2 Structure des notebooks

Partie I — MLP_Tabular_Classification.ipynb

Section	Contenu
1	Concepts fondamentaux (nn.Module, state_dict)
2	Chargement et préparation du dataset Breast Cancer Wisconsin
3	Implémentation des modèles (nn.Sequential et classe personnalisée)
4	Stratégies d'initialisation des poids
5	Entraînement et évaluation comparative
6	Sauvegarde et rechargement du meilleur modèle
7	Question de synthèse

Partie II — CNN_Image_Classification.ipynb

Section	Contenu
1	MLP vs CNN — pourquoi un MLP est inadapté aux images
2	Implémentation manuelle et validation des opérations convolutives
3	Chargement de MNIST et architectures (MLP, LeNet-5, CNN Amélioré)
4	Entraînement comparatif et étude architecturale (ablation)
5	Visualisation des feature maps
6	Évaluation finale sur le jeu de test et question de synthèse

Partie III — RNN_LSTM_GRU_Seq2Seq.ipynb

Section	Contenu
1	Fondements théoriques (modèle de langage, perplexité)
2	Vocabulaire, corpus et dataset char-level
3	Modèle récurrent générique (RNN/LSTM/GRU) et entraînement
4	Gradient clipping et génération de texte
5	Système Seq2Seq avec attention de Bahdanau
6	Décodage glouton, beam search, score BLEU et question de synthèse

6.3 Réponses aux questions de synthèse

- **Partie I — Le MLP est-il pertinent pour des données tabulaires réelles ?** Oui pour des données de taille modérée sans structure spatiale ou temporelle ; l'initialisation Xavier est déterminante pour la qualité de la convergence (~97,8 % accuracy).
- **Partie II — Pourquoi un CNN est-il plus pertinent qu'un MLP pour la classification d'images ?** Grâce à trois inductive biases — localité, partage des poids et hiérarchie des représentations — le CNN atteint une meilleure accuracy (~99,2 %) avec beaucoup moins de paramètres que le MLP.
- **Partie III — Comment justifier le passage du RNN simple au LSTM/GRU, puis au Seq2Seq ?** Le LSTM/GRU résout le problème du gradient qui disparaît grâce aux mécanismes de portes ; le Seq2Seq + attention permet en plus de gérer des séquences d'entrée et de sortie de longueurs différentes.

Références

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE.
- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation.
- Cho, K. et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation.
- Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural Machine Translation by Jointly Learning to Align and Translate.
- Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS.
- Documentation officielle PyTorch (pytorch.org) et scikit-learn (scikit-learn.org).